

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\`a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_e2a9aaa7-6b2\agent-acc61ba1678e769e8.jsonl`  
- SHA-256: `63297fb023b4ee6854ceb89f6237cb6e648b23fef4ab64bb346511f35625729f`  
- Source modified: `2026-05-30T19:13:42+00:00`  
- Imported at: `2026-07-08T15:59:25+00:00`  
- project: `wf_e2a9aaa7-6b2`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-30T19:12:54.812Z)

Source Extractor

Research question: "Find promising AND reliable open-source tools/libraries for parsing Indian bank statement PDFs ? especially HDFC Bank savings/transaction statements ? into structured transaction rows (date, narration, ref, value date, withdrawal, deposit, closing balance), for reuse in a LOCAL-FIRST Python personal-finance tool ("muneem").

HARD CONSTRAINTS (a tool that violates these is unusable for us):

1. Must run 100% LOCALLY. Bank statements are highly sensitive ? NO cloud APIs, NO paid SaaS (e.g. bankstatementconverter.com, docparser, LLM extraction APIs). Paid/cloud is not acceptable even as a default.
2. Must be usable from Python.
3. License MUST be permissive & commercially safe: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL or that depends on copyleft native components ? in particular Camelot/tabula's reliance on Ghostscript (AGPL) or a Java runtime. PyMuPDF (AGPL) is already banned for us.

Specifically evaluate and compare these table-extraction approaches, for each giving: license (AND license of native deps), maintenance status (latest release + activity as of 2025/2026), reliability on SEMI-RULED / partially-borderless tables (HDFC's transaction table is not fully line-ruled), and password-protected-PDF support:

- pdfplumber (we already use it; MIT)
- Camelot ? camelot-py vs the maintained "pypdf_table_extraction" fork; its Ghostscript/OpenCV deps
- tabula-py (tabula-java + JVM)
- pypdfium2-based table extraction
- any other notable local options

Also assess the user's idea: "convert the PDF table to Excel/CSV first, then parse" ? is an intermediate CSV/Excel extraction more reliable than direct coordinate/text parsing, and which LOCAL+permissive tools do PDF?CSV/Excel table conversion well?

Finally, search for any existing, maintained, permissively-licensed open-source projects specifically for parsing Indian bank statements (HDFC/ICICI/SBI/Axis) or bank statements generally, that we could reuse or learn from (GitHub/PyPI).

Deliver a ranked shortlist of the most promising + reliable + license-safe options for HDFC statement parsing in a local Python tool, each with its license (and native-dep licenses), maintenance status, strengths/weaknesses for this use case, and a concrete recommendation for muneem (reuse library X vs. hand-roll coordinate clustering with pdfplumber). Cite sources."

Fetch and extract key claims from this source:

****URL:**** <https://pypi.org/project/statementsparser/>

****Title:**** statementsparser ? PyPI (Indian bank statement parser: SBI/HDFC/ICICI/Axis)

****Found via:**** broad/primary ? library bake-off search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-30T19:13:00.178Z)

I'll fetch the PyPI page for statementsparser and extract relevant claims.

3. user (2026-05-30T19:13:12.792Z)

I cannot extract the requested details about a Python package for parsing Indian bank statements because the web page content provided does not contain any such information.

The page displays only an error message: "A required part of this site couldn't load. This may be due to a browser extension, network issues, or browser settings. Please check your connection, disable any ad blockers, or try using a different browser."

To answer your questions about supported banks, licensing, version numbers, dependencies, local/cloud operation, PDF password protection, output formats, and maintenance signals, I would need access to actual package documentation, a GitHub repository, or a working project page. The error message prevents me from retrieving any of these details.

4. assistant (2026-05-30T19:13:15.952Z)

The PyPI page returned a JavaScript error. Let me try the JSON API endpoint, which is a more reliable way to get PyPI package metadata.

5. user (2026-05-30T19:13:24.930Z)

StatementParser Package Analysis

1. Summary & Description

****Summary:**** "Parse Indian bank statements (SBI, HDFC, ICICI, Axis) into clean, unified pandas DataFrames"

****Supported Banks:**** The description states the package handles "SBI, HDFC, ICICI, Axis Bank" with a feature list noting "Multi-bank support ? SBI, HDFC, ICICI, Axis Bank"

****Key Capabilities:**** Auto-detects banks from PDF content, handles encrypted statements using date-of-birth passwords, extracts UPI metadata into structured fields, auto-categorizes transactions, verifies account balances, and exports to CSV/Excel/JSON formats.

2. License

****License Field:**** "MIT"

****Classifier:**** "License :: OSI Approved :: MIT License"

3. Version & Release Date

****Latest Version:**** "0.1.0"

****Upload Timestamps:****

- Wheel file: "2026-05-12T20:27:04"
- Source tarball: "2026-05-12T20:27:07"

4. Dependencies (requires_dist)

Core dependencies include: click, openpyxl, pandas, pdfplumber, pydantic, pymupdf, and rich. Development extras add pytest, mypy, and ruff.

5. Author & Project URLs

****Author:**** Harsh Lalakiya (harshlalakiya.me@gmail.com)

****GitHub Repository:**** <https://github.com/iharshlalakiya/statementsparser>

****Documentation:**** <https://iharshlalakiya.github.io/statementsparser>

6. Local Operation & Output Formats

The package operates locally via Python API or CLI tool (`bankparse`` command). It handles password-protected PDFs and supports output as CSV, Excel (with summary sheets), and JSON formats.

6. assistant (2026-05-30T19:13:29.722Z)

I have detailed metadata. The key findings are clear, including a critical license-dependency conflict relevant to muneem's constraints. Let me verify the PyMuPDF dependency detail since it's load-bearing for this evaluation.

The metadata is conclusive. The JSON API confirmed: MIT license, supports SBI/HDFC/ICICI/Axis, version 0.1.0 released 2026-05-12, runs locally, handles encrypted PDFs via DOB passwords, exports CSV/Excel/JSON ? but critically depends on `pymupdf`` (which muneem has explicitly banned due to AGPL licensing). This is a central finding for the research question.

7. user (2026-05-30T19:13:37.391Z)

Structured output provided successfully

8. assistant (2026-05-30T19:13:42.315Z)

Extracted the key claims from the statementsparser PyPI page. The standout finding for muneem: it is MIT-licensed and ticks nearly every box (HDFC/SBI/ICICI/Axis support, local operation, DOB-password handling, CSV/Excel/JSON output) but depends on `pymupdf`` (PyMuPDF) ? which is AGPL-licensed and explicitly banned in muneem's constraints. So despite the attractive surface, it is not directly reusable as-is, though it is highly instructive as prior art. It is also brand new (single 0.1.0 release dated 2026-05-12), so reliability is unproven.